SOFTWARE MAINTENANCE USING LLMS

Lab Report

BSSE1411 | SE-811

Institute of Information Technology (IIT) · University of Dhaka · 20 May 2026

Submitted To

Toukir Ahammed

Lecturer, IIT

University of Dhaka

LLM Models Evaluated

Claude Sonnet 4.5	Gemini 2.5 Pro	GPT-4o	GPT-5.2 Codex

TABLE OF CONTENTS

GitHub Link: *https://github.com/Rakibul1411/SE-811*

TASK 1 · SCROLL — Scrollbar and Canvas Rendering Bug

Task Name	SCROLL
Complaint	Scroll bars don't always appear after painting outside the canvas, and when they do appear the canvas doesn't look right.
Models Used	Claude Sonnet 4.5 Gemini 2.5 Pro
Prompt 1(Gemini 2.5 Pro)	"In this Java Swing paint app, users report that scrollbars don't appear after painting outside the canvas, and when they do appear the canvas looks wrong. Here is PaintCanvas.java: [code]. What is the bug and how should I fix it?"
Prompt 2(Claude Sonnet 4.5)	"In this Java Swing paint app, users report that scrollbars don't appear after painting outside the canvas, and when they do appear the canvas looks wrong. Here is PaintCanvas.java: [code]. What is the bug and how should I fix it?"

ROOT CAUSE ANALYSIS

Bug A – Canvas Background Misaligned: In `PaintCanvas.paintComponent()` the y-coordinate argument of `fillRect()` incorrectly used `getX()` instead of `getY()`. This caused the white background to be drawn at the wrong vertical offset when the canvas was scrolled, leaving visible artefacts.

Bug B – Scrollbars Never Appear: JScrollPane shows scrollbars only when the component's `preferredSize` exceeds the viewport. Since `setPreferredSize()` was called only once in the constructor and never updated afterward, the canvas always reported 800 × 600 even when objects were painted beyond those bounds.

CODE CHANGES — BUG A: FILLRECT Y-COORDINATE

✗ **BEFORE** (Original – Buggy Code)

```
// PaintCanvas.java  (~line 36)
g.fillRect((int)clipBounds.getX(), (int)clipBounds.getX(),  // ← BUG: X used for Y
           (int)clipBounds.getWidth(), (int)clipBounds.getHeight());
```

✓ **AFTER** (Fixed Code)

```
// PaintCanvas.java  (~line 36)
g.fillRect((int)clipBounds.getX(), (int)clipBounds.getY(),  // ✓ fixed: Y for Y
           (int)clipBounds.getWidth(), (int)clipBounds.getHeight());
```

CODE CHANGES — BUG B: CANVAS PREFERRED SIZE EXPANSION

✓ **AFTER** (Fixed Code)

```
// NEW helper method in PaintCanvas.java
private void expandCanvasToFit(PaintObject obj) {
    Rectangle bbox = obj.getBoundingBox();
    Dimension current = getPreferredSize();
    int newWidth  = Math.max(current.width,  bbox.x + bbox.width  + 20);
    int newHeight = Math.max(current.height, bbox.y + bbox.height + 20);
    if (newWidth != current.width || newHeight != current.height) {
        setPreferredSize(new Dimension(newWidth, newHeight));
        revalidate();   // notify JScrollPane
    }
}

// MODIFIED: addPaintObject() — call helper after adding object
public void addPaintObject(PaintObject newObject) {
    history.addElement(new Vector(paintObjects));
    paintObjects.addElement(newObject);
    expandCanvasToFit(newObject);   // ← new call
    repaint();
}
```

FILES & LINES CHANGED

File	Change Description	Lines
PaintCanvas.java	fillRect argument fix (getX → getY)	1 modified
PaintCanvas.java	Import: javax.swing.SwingUtilities	+1
PaintCanvas.java	New expandCanvasToFit() method	+10
PaintCanvas.java	Call expandCanvasToFit() in addPaintObject()	+1
TOTAL		**~13 lines**

VERIFICATION RESULTS

- Compiled with javac edu/cmu/hcii/paint/*.java — zero errors.
- Painted strokes near and beyond the right/bottom edges — scrollbars appeared immediately.
- Scrolled the view — white background rendered correctly, no artefacts.

MODEL COMPARISON

Model	Bug A (fillRect)	Bug B (preferred size)	Code Quality
Gemini 2.5 Pro	Yes	Partial — mentioned, no code	Good — needed a follow-up prompt for size expansion
Claude Sonnet 4.5	Yes	Yes — full implementation	Excellent — both bugs fixed in one response, compilable code

Best Solution: **Claude Sonnet 4.5** — Identified both bugs and delivered complete, ready-to-use code without a refinement prompt.

TASK 2 · YELLOW — Yellow Colour Cannot Be Selected

Task Name	YELLOW
Complaint	Users cannot select yellow. The colour shown for R=255, G=255, B=0 is not yellow.
Models Used	**Claude Sonnet 4.5** **GPT-4o** **GPT-5.2 Codex**
Prompt 1(GPT-4o)	"Scan PaintWindow.java for variable-aliasing bugs in the color construction call and suggest the minimal fix."
Prompt 2(Claude Sonnet 4.5)	"Scan PaintWindow.java for variable-aliasing bugs in the color construction call and suggest the minimal fix."
Prompt 3(GPT-5.2 Codex)	"Scan PaintWindow.java for variable-aliasing bugs in the color construction call and suggest the minimal fix."

ROOT CAUSE ANALYSIS

The blue channel in `colorChangeListener` mistakenly referenced `gSlider` instead of `bSlider`. This meant the blue value always equalled the green value, collapsing the colour space so that only colours where B = G could ever be produced — ruling out yellow, cyan, red, orange, and most other colours.

CODE CHANGES

✗ **BEFORE** (Original – Buggy Code)

```
// PaintWindow.java — inside anonymous ChangeListener
objectConstructor.setColor(
    new Color(rSlider.getValue(),
              gSlider.getValue(),
              gSlider.getValue())   // ← BUG: g used instead of b
```

✓ **AFTER** (Fixed Code)

```
// PaintWindow.java — inside anonymous ChangeListener
objectConstructor.setColor(
    new Color(rSlider.getValue(),
              gSlider.getValue(),
              bSlider.getValue())   // ✓ fixed: correct blue channel
```

FILES & LINES CHANGED

File	Change Description	Lines
PaintWindow.java	Replace gSlider.getValue() → bSlider.getValue() for blue channel	1 modified

VERIFICATION RESULTS

- R=255, G=255, B=0 → colour swatch showed yellow.
- R=255, G=0, B=0 → red.
- R=0, G=0, B=255 → blue.
- R=128, G=0, B=128 → purple. (previously impossible)

MODEL COMPARISON

Model	Found Bug	Explanation Quality	Notes
GPT-4o	Yes	Excellent	Immediately spotted the duplicate gSlider reference
Claude Sonnet 4.5	Yes	Excellent	Also enumerated the full set of unreachable colours
GPT-5.2 Codex	Yes	Excellent	Found the alias via static analysis framing; clean one-liner fix

Best Solution: All three models — Tie — Each model identified the single-character bug immediately. Claude provided the richest explanation of the colour-space impact.

TASK 3 · UNDO — Undo Button Does Not Always Work

Task Name	UNDO
Complaint	The "Undo my last stroke" button doesn't always work — sometimes it has no visible effect, and sometimes it crashes the app on an empty canvas.
Models Used	Claude Sonnet 4.5 Gemini 2.5 Pro
Prompt 1(Gemini 2.5 Pro)	"In this Swing paint app the Undo button sometimes has no visible effect. Relevant code: PaintCanvas.undo(), PaintWindow.undo(), Actions.java. What is wrong and how do I fix it?"
Prompt 2(Claude Sonnet 4.5)	"Identify all bugs in the undo functionality across PaintCanvas.java, PaintWindow.java, and Actions.java, and provide fixed versions."

ROOT CAUSE ANALYSIS

Bug A – No repaint() after undo: `PaintCanvas.undo()` restored the data model correctly but never called `repaint()`. The canvas was not redrawn, so the undone stroke remained visually present even though the internal state was correct.

Bug B – Undo button active before first stroke: Actions.java created undoAction without calling `setEnabled(false)`. Clicking it on an empty canvas caused a `NoSuchElementException` crash when `history.lastElement()` was called on an empty vector.

CODE CHANGES — BUG A: MISSING REPAINT()

✗ **BEFORE** (Original – Buggy Code)

```
// PaintCanvas.java
public void undo() {
    paintObjects = (Vector) history.lastElement();
    history.removeElement(history.lastElement());
    // ← BUG: no repaint() — canvas not redrawn
}
```

✓ **AFTER** (Fixed Code)

```
// PaintCanvas.java
public void undo() {
    if (history.isEmpty()) return;   // ✓ guard against empty history
    paintObjects = (Vector) history.lastElement();
    history.removeElement(history.lastElement());
    repaint();   // ✓ trigger canvas redraw
}
```

CODE CHANGES — BUG B: BUTTON ENABLED TOO EARLY

✗ **BEFORE** (Original – Buggy Code)

```
// Actions.java — undoAction setup
undoAction.putValue(Action.NAME, "Undo my last stroke");
// ← BUG: button is enabled at launch, can crash on empty canvas
```

✓ **AFTER** (Fixed Code)

```
// Actions.java — undoAction setup
undoAction.putValue(Action.NAME, "Undo my last stroke");
undoAction.setEnabled(false);   // ✓ disabled until first stroke completes
```

FILES & LINES CHANGED

File	Change Description	Lines
PaintCanvas.java	Guard: if(history.isEmpty()) return;	+1
PaintCanvas.java	Added: repaint() at end of undo()	+1
Actions.java	Added: undoAction.setEnabled(false)	+1
TOTAL		**3 lines**

VERIFICATION RESULTS

- On launch — Undo button is greyed out.
- After drawing one stroke — button becomes enabled.
- Clicking Undo — last stroke removed visually.
- After undoing all strokes — button becomes greyed out again.
- Clicking Undo on an empty canvas — no crash, silent guard.

MODEL COMPARISON

Model	Bug A (no repaint)	Bug B (enabled too early)	Notes
Gemini 2.5 Pro	Yes	No	Missed the enabled-too-early issue entirely
Claude Sonnet 4.5	Yes	Yes	Found both bugs; provided all fixes in one response

Best Solution: **Claude Sonnet 4.5** — Only model to find both bugs. Gemini found only Bug A.

TASK 4 · LINE — Line Tool Not Functional

Task Name	LINE
Complaint	A Line radio button exists in the UI but does nothing when selected — no line is drawn.
Models Used	**Claude Sonnet 4.5** **GPT-4o** **GPT-5.2 Codex**
Prompt 1(GPT-4o)	"Generate a complete LinePaint.java extending PaintObject for straight-line drawing, wire it to a new lineAction in Actions.java, and fix the lineButton constructor in PaintWindow.java."
Prompt 2(Claude Sonnet 4.5)	"Generate a complete LinePaint.java extending PaintObject for straight-line drawing, wire it to a new lineAction in Actions.java, and fix the lineButton constructor in PaintWindow.java."
Prompt 3(GPT-5.2 Codex)	"Generate a complete LinePaint.java extending PaintObject for straight-line drawing, wire it to a new lineAction in Actions.java, and fix the lineButton constructor in PaintWindow.java."

ROOT CAUSE ANALYSIS

Two compounding problems:

- **No ActionListener:** lineButton was constructed as `new JRadioButton("Line")` — no action attached, so selecting it never called `setPaintObjectClass()`
- **No LinePaint class:** Even if the button had been wired, there was no `LinePaint` class — the runtime would have thrown a class-not-found error.

CODE CHANGES — BUTTON WIRING

✗ **BEFORE** (Original – Buggy Code)

```
// PaintWindow.java
lineButton = new JRadioButton("Line");    // ← no action attached
```

✓ **AFTER** (Fixed Code)

```
// PaintWindow.java
lineButton = new JRadioButton(actions.lineAction);    // ✓ wired to action
```

CODE CHANGES — NEW LINEACTION (ACTIONS.JAVA)

✓ **AFTER** (Fixed Code)

```
// Actions.java — new field and action block
public AbstractAction lineAction;

lineAction = new AbstractAction() {
    public void actionPerformed(ActionEvent actionEvent) {
        paintWindow.setPaintObjectClass(LinePaint.class);
    }
};
lineAction.putValue(Action.NAME, "Line");
```

NEW FILE — LINEPAINT.JAVA (KEY METHODS)

\+ **NEW FILE: LinePaint.java**

```
// edu/cmu/hcii/paint/LinePaint.java
public class LinePaint extends PaintObject {
    private Point start, end;

    public void define(Point[] points) {
        if (points == null || points.length == 0) return;    // null guard
        start = points[0];
        end   = points[points.length - 1];
    }

    public Rectangle getBoundingBox() {
        int x = Math.min(start.x, end.x) - thickness / 2;
        int y = Math.min(start.y, end.y) - thickness / 2;
        int w = Math.abs(end.x - start.x) + thickness;
        int h = Math.abs(end.y - start.y) + thickness;
        return new Rectangle(x, y, w, h);
    }

    public void paint(Graphics2D g) {
        if (start == null || end == null) return;
        Stroke oldStroke = g.getStroke();
        g.setStroke(new BasicStroke(thickness,
                    BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
        g.setColor(color);
        g.drawLine(start.x, start.y, end.x, end.y);
        g.setStroke(oldStroke);
    }
}
```

FILES & LINES CHANGED

File	Change Description	Lines
LinePaint.java (NEW)	Complete new PaintObject subclass for straight-line drawing	+23
Actions.java	Field declaration + lineAction block	+7
PaintWindow.java	lineButton constructor (1 line replaced)	1 modified
TOTAL		**~31 lines**

VERIFICATION RESULTS

- Selected the Line radio button and dragged the mouse — live preview line appeared while dragging.
- Released the mouse — final line committed to canvas.
- Undo correctly removed the line.
- Line colour and thickness respected slider values.

MODEL COMPARISON

Model	LinePaint.java	Actions + Button	Null Guards	Notes
GPT-4o	Yes	Yes	No	Functional code; used CAP_ROUND; no defensive checks
Claude Sonnet 4.5	Yes	Yes	Yes	Added null guard in define(); prevents NPE on hover prototype
GPT-5.2 Codex	Yes	Yes	Yes	Added null guard and also provided Javadoc; cleanest code structure

Best Solution: **Claude Sonnet 4.5 / GPT-5.2 Codex** — Both added null guards absent from GPT-4o. Codex produced the most structured code; Sonnet required fewest follow-up prompts.

TASK 5 · THICKNESS — Stroke Thickness Control

Task Name	THICKNESS
Request	Add a UI control for stroke thickness (pencil, eraser, line tools). Also fix EraserPaint which ignores the thickness parameter.
Models Used	Claude Sonnet 4.5 Gemini 2.5 Pro GPT-5.2 Codex
Prompt 1(Gemini 2.5 Pro)	"Fix the hardcoded thickness bug in EraserPaint.java and generate a full thickness-slider implementation for PaintWindow.java including GridBagLayout registration."
Prompt 2(Claude Sonnet 4.5)	"Fix the hardcoded thickness bug in EraserPaint.java and generate a full thickness-slider implementation for PaintWindow.java including GridBagLayout registration."
Prompt 3(GPT-5.2 Codex)	"Fix the hardcoded thickness bug in EraserPaint.java and generate a full thickness-slider implementation for PaintWindow.java including GridBagLayout registration."

ROOT CAUSE ANALYSIS

Bug — EraserPaint ignores setThickness(): The method body assigned the constant `25` directly to `this.thickness`, shadowing the parameter. Eraser size was always 25 px regardless of user input.

Feature Gap — No UI Slider: The initial thickness was set to 5 in the constructor via `objectConstructor.setThickness(5)`, but there was no widget in the control panel to change it at runtime.

CODE CHANGES — ERASERPAINT BUG FIX

✗ **BEFORE** (Original – Buggy Code)

```
// EraserPaint.java
public void setThickness(int thickness) {
    this.thickness = 25;    // ← BUG: parameter ignored, hardcoded constant
}
```

✓ **AFTER** (Fixed Code)

```
// EraserPaint.java
public void setThickness(int thickness) {
    this.thickness = thickness;    // ✓ fixed: use the parameter
}
```

CODE CHANGES — THICKNESS SLIDER (PAINTWINDOW.JAVA)

✓ **AFTER** (Fixed Code)

```
// PaintWindow.java — new fields
private JPanel  thicknessPanel;
private JSlider thicknessSlider;

// PaintWindow.java — slider construction (after colorPanel setup)
thicknessPanel = new JPanel(new FlowLayout());
thicknessPanel.setOpaque(false);
thicknessPanel.add(new JLabel("Thickness"));

thicknessSlider = new JSlider(1, 50, 5);
thicknessSlider.setOpaque(false);
thicknessSlider.addChangeListener(new javax.swing.event.ChangeListener() {
    public void stateChanged(javax.swing.event.ChangeEvent e) {
        objectConstructor.setThickness(thicknessSlider.getValue());
    }
});
thicknessPanel.add(thicknessSlider);

// GridBagLayout registration (between colorPanel and clear/undo panel)
controlPanel.add(thicknessPanel);
```

FILES & LINES CHANGED

File	Change Description	Lines
EraserPaint.java	Replace this.thickness = 25 with this.thickness = thickness	1 modified
PaintWindow.java	Field declarations (2 fields)	+2
PaintWindow.java	Slider + panel construction + ChangeListener	+11
PaintWindow.java	GridBag constraints + add to controlPanel	+1
TOTAL		**~15 lines**

VERIFICATION RESULTS

- Thickness slider set to 1 — very thin stroke drawn.
- Thickness slider set to 30 — thick stroke drawn.
- Switched to Eraser; slider set to 10 — eraser used 10 px width.
- Switched to Line tool — thickness respected correctly.

MODEL COMPARISON

Model	EraserPaint Fix	Slider Added	GridBag Registration	Notes
Gemini 2.5 Pro	Yes	Yes	No	Omitted GridBag constraint registration — caused layout issues
Claude Sonnet 4.5	Yes	Yes	Yes	Complete, layout-correct solution in one shot
GPT-5.2 Codex	Yes	Yes	Yes	Complete solution; also suggested setMajorTickSpacing() for usability

Best Solution: **Claude Sonnet 4.5 / GPT-5.2 Codex** — Both produced full solutions; Gemini required a follow-up fix for GridBag registration.

SUMMARY & OVERALL OBSERVATIONS

Summary Table

#	Task	Root Cause	Files Changed	Lines	Best Model
1	SCROLL	getX() used as Y in fillRect; preferredSize never updated	PaintCanvas.java	~13	Claude Sonnet 4.5
2	YELLOW	Blue channel referenced gSlider instead of bSlider	PaintWindow.java	1	Three-way tie
3	UNDO	Missing repaint(); button enabled before first stroke	PaintCanvas.java, Actions.java	3	Claude Sonnet 4.5
4	LINE	No ActionListener on button; LinePaint class missing	LinePaint.java (new), Actions.java, PaintWindow.java	~31	Sonnet 4.5 / Codex
5	THICKNESS	EraserPaint hardcoded 25; no UI slider in control panel	EraserPaint.java, PaintWindow.java	~15	Sonnet 4.5 / Codex

Model Performance Overview

Model	Tasks Tested	Bugs Found (all)	Complete Solution
Claude Sonnet 4.5	5 / 5	10 / 10	5 / 5
Gemini 2.5 Pro	3 / 5	5 / 6	2 / 3
GPT-4o	2 / 5	3 / 3	2 / 2
GPT-5.2 Codex	3 / 5	6 / 6	3 / 3

Key Observations

- **Claude Sonnet 4.5 (Anthropic):** Consistently delivered the most complete solutions across all five tasks. Unique in finding both UNDO bugs simultaneously and including defensive null-checks in LinePaint without prompting. Zero follow-up prompts needed.
- **GPT-5.2 Codex (OpenAI):** Matched Sonnet 4.5 on every task it was tested on, and produced the most structured, Javadoc-commented code. Particularly strong at static-analysis-style bug detection (YELLOW aliasing).
- **GPT-4o (OpenAI):** Highly accurate on simpler, single-bug tasks (YELLOW, LINE). Competitive with Sonnet on well-scoped prompts but did not add defensive guards proactively.
- **Gemini 2.5 Pro (Google):** Successfully identified the primary symptom in every tested task but missed secondary bugs (UNDO Bug B) and omitted boilerplate integration code (GridBag registration in THICKNESS), requiring follow-up prompts.
- **Prompt quality matters:** All four models performed significantly better when given exact relevant source code alongside a precise symptom description. Vague prompts led to partial or generic answers.
- **LLMs as maintenance tools:** All models successfully identified bugs that a developer might spend hours tracking down manually, demonstrating strong value for targeted software maintenance tasks when combined with structured prompting.